

👉 XAI: LIME & SHAP

FAMAF - UNC

First Semester 2026



Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

<https://interpretable-ml-class.github.io/>

“Why Should I Trust You?” Explaining the Predictions of Any Classifier

Marco Tulio Ribeiro
University of Washington
Seattle, WA 98105, USA
marcotcr@cs.uw.edu

Sameer Singh
University of Washington
Seattle, WA 98105, USA
sameer@cs.uw.edu

Carlos Guestrin
University of Washington
Seattle, WA 98105, USA
guestrin@cs.uw.edu

Aug 2016

ABSTRACT

Despite widespread adoption, machine learning models remain mostly black boxes. Understanding the reasons behind predictions is, however, quite important in assessing *trust*, which is fundamental if one plans to take action based on a prediction, or when choosing whether to deploy a new model.

how much the human understands a model's behaviour, as opposed to seeing it as a black box.

Determining trust in individual predictions is an important problem when the model is used for decision making. When using machine learning for medical diagnosis [6] or terrorism detection, for example, predictions cannot be acted upon on blind faith, as the consequences may be catastrophic.

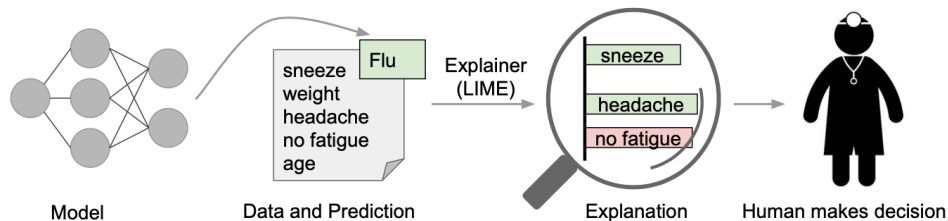
(Ribeiro, Singh, and Guestrin 2016)

🍲 Define Trust - Two Notions ---

ML models are often used as black boxes — but humans need to understand them before acting on their outputs.

- **Trusting a prediction:** Does this specific output make sense? *E.g., should a doctor act on this diagnosis?*
- **Trusting a model:** Will this model behave reasonably when deployed? *E.g., does it generalize beyond the validation set?*
Both depend on how much the user understands the model's behavior.

🍷 Define Explanation for a Prediction ---



An explanation identifies **which parts of the input** drove the model's prediction — and in which direction.

- ● *sneeze, headache* → evidence **for** Flu
- ● *no fatigue* → evidence **against** Flu

Formally:

A textual or visual artifact that describes the relationship between an instance's components and the model's output.

🍷 Explanations as a Means to Select Model 1/2 ---

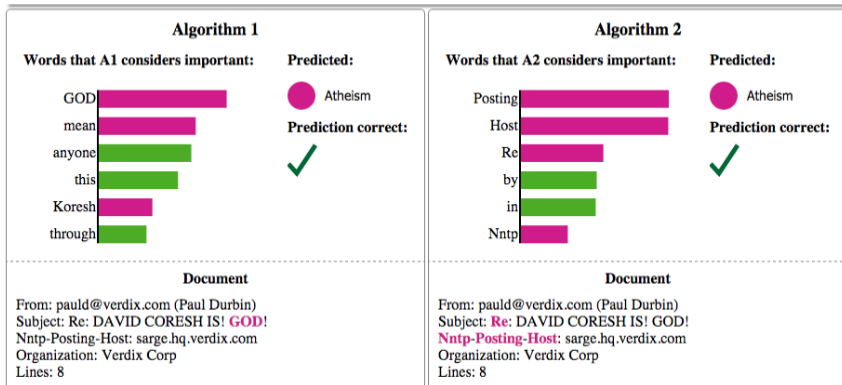
Example #3 of 6

True Class:  Atheism

[Instructions](#)

[Previous](#)

[Next](#)



Both models predict "Atheism" correctly — but for very different reasons.

🍷 Explanations as a Means to Select Model 2/2 ---

Both models predict "**Atheism**" correctly — but for very different reasons.

	Algorithm 1	Algorithm 2
Key words	<i>GOD, mean, anyone</i>	<i>Posting, Host, Nntp</i>
Reason	Semantic content ✓	Email header metadata ✗
Will it generalize?	Likely yes	Likely no

- Accuracy alone cannot distinguish these models.
- Explanations expose *why* a prediction was made — and whether to trust it.

🍷 Why Do We Need Trust and Interpretability? ---

A model can have **high accuracy** and still be wrong for the right reasons.

- **Data leakage:** the model learns features that are accidentally correlated with the label (e.g., patient ID predicts diagnosis). High accuracy, zero generalization.
- **Dataset shift / Concept drift:** training and real-world data differ — either the input distribution changes/shift, or the relationship between inputs and labels changes/drift over time. The model looks good on validation but fails in deployment.
- **Spurious features:** the model exploits features that *work* statistically but are undesirable (e.g., clickbait signals in a recommender system).
In all three cases, validation accuracy is misleading. Interpretability lets us catch these failures before they cause harm.

🍷 Desired Characteristics for Explainers ---

- **Interpretable:** explanations must be understandable to humans, not just technically correct (e.g., no thousands of non-zero weights)
- **Locally faithful:** must reflect how the model *actually behaves* near the instance being explained — global fidelity is often impossible
- **Model-agnostic:** must work as a black box, applicable to any classifier, including models that don't yet exist
- **Global perspective:** beyond single predictions, explanations should help users understand the model as a whole

Local Interpretable Model-agnostic Explanations (LIME)

A framework designed to satisfy all four criteria simultaneously.

🍷 LIME Step 1: Define LIME Framework ---

Goal: Find the simplest explanation that still accurately describes the model's behavior *near* the instance x .

$$\xi(x) = \underset{g \in G}{\operatorname{argmin}} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

- $\xi(x)$ — explanation for instance x
- $g \in G$ — interpretable model (e.g., linear model)
- f — black-box model to be explained
- $\mathcal{L}(f, g, \pi_x)$ — unfaithfulness of g approximating f locally
- π_x — proximity measure that defines the local neighborhood
- $\Omega(g)$ — complexity of g (e.g., number of non-zero weights)

This is a **fidelity-interpretability trade-off**: minimize error locally, while keeping the explanation simple enough for humans to understand.

🍷 LIME Step 2: Interpretable Data Representation ---

- The black-box model operates on features that are often incomprehensible to humans (e.g., embeddings, raw pixels).
- LIME maps these to an **interpretable representation** that humans can reason about:
 - ▶ **Text:** binary vector indicating the presence or absence of a word
 - ▶ **Image:** binary vector indicating the presence or absence of a contiguous patch of similar pixels (superpixel)

$$x \in \mathbb{R}^d \xrightarrow{h_x} x' \in \{0, 1\}^{d'}$$

Original Representation $\longrightarrow$ Interpretable Representation

where $x \in \mathbb{R}^d$ is the original instance with d features, $x' \in \{0, 1\}^{d'}$ is a binary vector of length d' where each entry is 1 if the corresponding interpretable component is **present** and 0 if it is **absent**, and h_x is the function that maps x' back to the original representation.

🍷 LIME Step 3: Sampling for Local Exploration 1/2 ---

Since f is a black box, LIME **interrogates** it: perturb, query, and fit g locally.

- 1 Start from x' , the interpretable representation of the instance to explain
- 2 Generate perturbed samples z' by randomly turning off components of x'
- 3 Map each z' back to the original space: $z = h_x(z')$
- 4 Query the black-box model to get a label: $f(z)$
- 5 Weight each sample by its proximity to x : $\pi_x(z)$

$$x \rightarrow x' \xrightarrow{\text{perturb}} z' \xrightarrow{h_x} z \xrightarrow{f} f(z)$$

The dataset $\mathcal{Z} = \{(z', f(z), \pi_x(z))\}$ is then used to fit the local model g .

LIME Step 3: Sampling for Local Exploration 2/2 ---

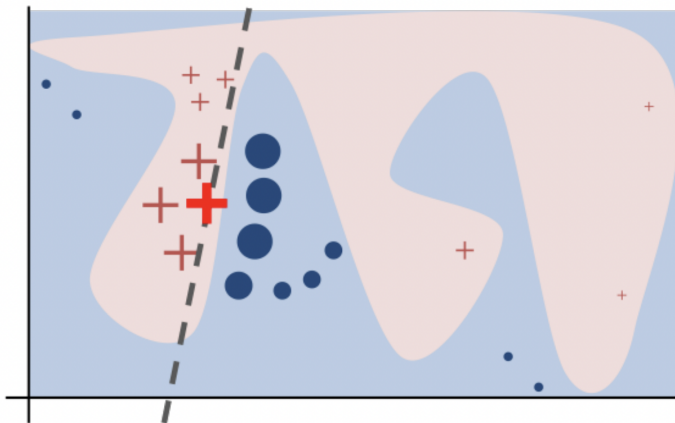
Since f is a black box, LIME cannot minimize $\mathcal{L}(f, g, \pi_x)$ analytically. Instead, it **interrogates** f : it generates perturbed samples near x' , queries the model, and uses the responses to fit g locally.

- ➊ **Start from the instance:** take the review “*The movie was great and funny*” $\rightarrow x' = [1, 1, 1, 1, 1, 1]$
- ➋ **Randomly turn off words:** generate a masked version $\rightarrow z' = [0, 1, 0, 1, 0, 1] \rightarrow$ “*movie great funny*”
- ➌ **Map back to original space:** reconstruct the input the model understands $\rightarrow z = h_x(z')$ (e.g., recompute embeddings for “*movie great funny*”)
- ➍ **Ask the black box:** what does f predict for this masked input? $\rightarrow f(z) = 0.85$ (positive sentiment)
- ➎ **Weight by proximity:** samples closer to x matter more $\rightarrow \pi_x(z)$ is high if few words were removed, low otherwise

$$x \rightarrow x' \xrightarrow{\text{perturb}} z' \xrightarrow{h_x} z \xrightarrow{f} f(z)$$

Repeat N times $\rightarrow$ fit a sparse linear model g on $\mathcal{Z} = \{(z', f(z), \pi_x(z))\}$.

🍷 Sampling Procedure Intuition ---



A complex decision boundary is approximated locally by a simple linear model (dashed line).
Samples closer to the instance of interest (red cross) are weighted more.

🍷 Example: Sparse Linear Explanation ---

$$\xi(x) = \operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

$$g(z') = w_g \cdot z'$$

$$\pi_x(z) = \exp\left(-\frac{D(x, z)^2}{\sigma^2}\right)$$

$$\mathcal{L}(f, g, \pi_x) = \sum_{z, z' \in \mathcal{Z}} \pi_x(z) (f(z) - g(z'))^2$$

$$\Omega(g) = \infty \mathbb{I}[\|w_g\|_0 > K]$$

Algorithm 1 Sparse Linear Explanations using LIME

Require: Classifier f , Number of samples N

Require: Instance x , and its interpretable version x'

Require: Similarity kernel π_x , Length of explanation K

$\mathcal{Z} \leftarrow \{\}$

for $i \in \{1, 2, 3, \dots, N\}$ **do**

$z'_i \leftarrow \text{sample_around}(x')$

$\mathcal{Z} \leftarrow \mathcal{Z} \cup \langle z'_i, f(z_i), \pi_x(z_i) \rangle$

end for

$w \leftarrow \text{K-Lasso}(\mathcal{Z}, K) \triangleright$ with z'_i as features, $f(z)$ as target

return w

🍷 Example: Sparse Linear Explanation 🐍 ---

$$\xi(x) = \operatorname{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

$$g(z') = w_g \cdot z'$$

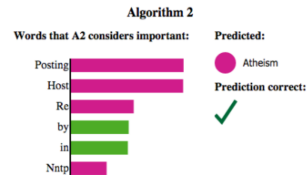
$$\pi_x(z) = \exp\left(-\frac{D(x, z)^2}{\sigma^2}\right)$$

$$\mathcal{L}(f, g, \pi_x) = \sum_{z, z' \in \mathcal{Z}} \pi_x(z) (f(z) - g(z'))^2$$

$$\Omega(g) = \infty \mathbb{I}[\|w_g\|_0 > K]$$

```
def lime_explain(f, x, x_prime, pi_x, K, N):  
    """  
    f      : black-box model (callable)  
    x      : original instance  
    x_prime : interpretable representation of x (binary vector)  
    pi_x   : proximity function centered at x  
    K      : max number of features in explanation  
    N      : number of samples  
    """  
    Z = []  
    for _ in range(N):  
        # Step 1: perturb x' by randomly turning off components  
        z_prime = random_mask(x_prime)  
        # Step 2: map back to original representation  
        z = h_x(z_prime, x)  
        # Step 3: query the black-box model  
        label = f(z)  
        # Step 4: compute proximity weight  
        weight = pi_x(z)  
        Z.append((z_prime, label, weight))  
    # Step 5: fit sparse linear model with at most K features  
    g = k_lasso(Z, K)  
    return g
```


🍷 Some Results ---



Document

From: pauld@verdix.com (Paul Durbin)
Subject: **Re:** DAVID CORESH IS! GOD!
Nntp-Posting-Host: sarge.hq.verdix.com
Organization: Verdix Corp
Lines: 8

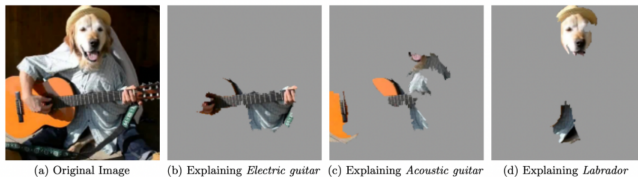


Figure 4: Explaining an image classification prediction made by Google's Inception neural network. The top 3 classes predicted are "Electric Guitar" ($p = 0.32$), "Acoustic guitar" ($p = 0.24$) and "Labrador" ($p = 0.21$)

🍷 Global LIME: Intuition ---

If we could explain **every instance** in the dataset, we would have a complete picture of how the model behaves globally.

But this is clearly infeasible — datasets can have thousands of instances, and users have limited time and patience.

Key question:

Which B instances should we show the user to maximize their understanding of the model?

🍷 Gain Global Understanding of the Model ---

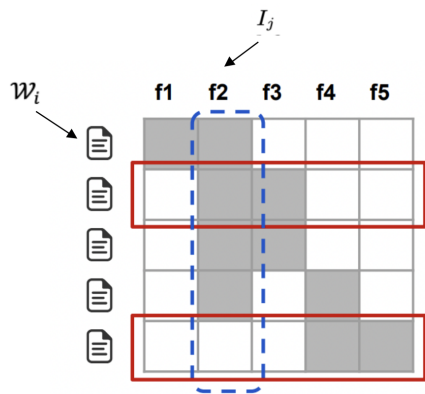
A single explanation gives local insight — but how do we understand the model **globally**? The pick step selects B instances to show the user, such that together they are maximally informative about the model's overall behavior.

- **Budget B :** users have limited time — we can only show B explanations
- **Explanation-aware selection:** instances are chosen based on their LIME explanations, not raw data alone — looking at raw predictions is not enough
- **Diverse and representative:** selected instances should cover as many different important features as possible, avoiding redundancy

The goal: with just B explanations, give the user a global picture of how the model works.

🍷 Submodular Pick (SP) Algorithm 1/2 ---

After running LIME on every B instance, we organize the results into a matrix $\mathcal{W}_{n \times d'}$ that summarizes how important each interpretable feature is across all instances.



- Each **row** i represents an instance x_i
- Each **column** j represents an interpretable feature (e.g., a word)
- Each **cell** $\mathcal{W}_{ij} = |w_{g_j}|$ is the importance LIME assigned to feature j for instance i — 0 if unused

From $\mathcal{W}$, the **global importance** of each feature is:

$$I_j = \sqrt{\sum_{i=1}^n |\mathcal{W}_{ij}|}$$

A high I_j means feature j is relevant across many instances.

Submodular Pick (SP) Algorithm 2/2 ---

Algorithm 2 Submodular pick (SP) algorithm

Require: Instances X , Budget B

```
for all  $x_i \in X$  do
     $\mathcal{W}_i \leftarrow \text{explain}(x_i, x'_i)$  ▷ Using Algorithm 1
end for

for  $j \in \{1 \dots d'\}$  do
     $I_j \leftarrow \sqrt{\sum_{i=1}^n |\mathcal{W}_{ij}|}$  ▷ Compute feature importances
end for

 $V \leftarrow \{\}$ 
while  $|V| < B$  do ▷ Greedy optimization of Eq (4)
     $V \leftarrow V \cup \operatorname{argmax}_i c(V \cup \{i\}, \mathcal{W}, I)$ 
end while
return  $V$  Pick( $\mathcal{W}, I$ )
```

We want to select a set V of at most B instances that **maximizes coverage** of globally important features:

$$c(V, \mathcal{W}, I) = \sum_{j=1}^{d'} \mathbb{1}_{[\exists i \in V: \mathcal{W}_{ij} > 0]} I_j$$

$$\text{Pick}(\mathcal{W}, I) = \operatorname{argmax}_{V, |V| \leq B} c(V, \mathcal{W}, I)$$

c is a **submodular function**: adding a new instance to V yields diminishing returns as V grows.

Greedy solution: at each step, add the instance that maximally increases coverage:

Simulated User Experiment - Experimental Setup ---

Goal: evaluate explanation quality across different models and methods.

Datasets: two sentiment analysis datasets (books and DVDs, 2000 instances each) — task is to classify product reviews as positive or negative.

Models trained: Decision Tree, Logistic Regression, Nearest Neighbors, SVM, Random Forest

Explanation methods compared:

- **Random:** selects K features at random — minimal baseline
- **Greedy:** removes features one by one until the prediction changes
- **Parzen:** approximates f globally with Parzen windows, explains via gradient
- **LIME:** proposed method — local sparse linear approximation

Instance selection strategies:

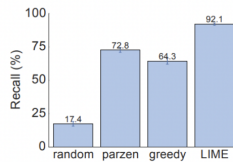
- **Random Pick (RP):** selects instances at random
- **Submodular Pick (SP):** selects instances to maximize feature coverage

All methods produce explanations of size $K = 10$ features.

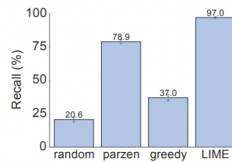
🍷 Simulated User Experiment - Are explanations faithful to the method? ---

Key idea: use interpretable models (sparse LR, decision trees) where we *know* the true important features — the **gold set**. Then measure how many gold features each method recovers (**recall**).

LIME achieves the highest recall of truly important features on both Books and DVDs datasets.

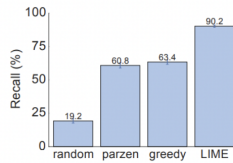


(a) Sparse LR

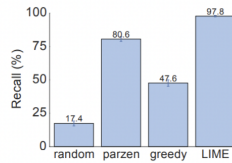


(b) Decision Tree

Figure 6: Recall on truly important features for two interpretable classifiers on the books dataset.



(a) Sparse LR



(b) Decision Tree

Figure 7: Recall on truly important features for two interpretable classifiers on the DVDs dataset.

🍷 Simulated User Experiment - Should I trust this prediction? ---

- Randomly mark 25% of features as untrustworthy (simulating problematic features such as data leakage or spurious correlations). A prediction is labeled **untrustworthy** if it changes when those features are removed (the model was relying on them).
- The prediction is then untrustworthy if the problematic features appear in it with high weight.
- We measure **F1 of trustworthiness** to evaluate whether each explanation method correctly identifies which predictions to trust and which to reject.

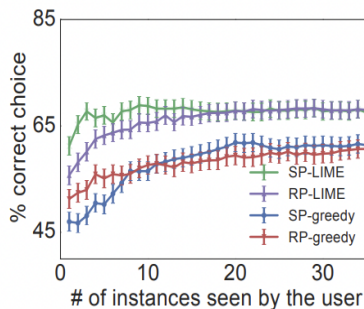
Table 1: Average F1 of *trustworthiness* for different explainers on a collection of classifiers and datasets.

	Books				DVDs			
	LR	NN	RF	SVM	LR	NN	RF	SVM
Random	14.6	14.8	14.7	14.7	14.2	14.3	14.5	14.4
Parzen	84.0	87.6	94.3	92.3	87.0	81.7	94.2	87.3
Greedy	53.7	47.4	45.0	53.3	52.4	58.1	46.6	55.1
LIME	96.6	94.5	96.2	96.7	96.6	91.8	96.1	95.6

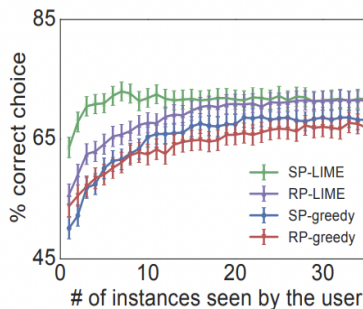
LIME maintains both high precision and high recall — it neither over-trusts nor over-distrusts predictions.

🍷 Simulated User Experiment - Can I trust this model? ---

- 10 noisy features added: correlated with the label in train/validation, but not in test
- Two random forests with similar validation accuracy ($< 0.1\%$ apart) but very different test accuracy ($> 5\%$ apart)
- **Task:** can a simulated user pick the better model by inspecting B explanations?
- **SP-LIME** dominates, especially for small B



(a) Books dataset



(b) DVDs dataset

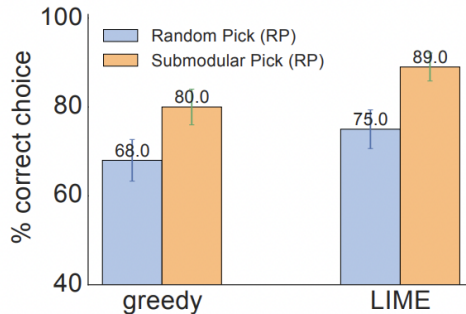
🍷 Evaluating with Human Subjects - Experimental Setup ---

Previous experiments were simulated — now we test with **real users** on Amazon Mechanical Turk.

- **Dataset:** 20 Newsgroups (Christianity vs. Atheism) — known to contain spurious features (headers, author names) that do not generalize
- To measure real-world generalization, a **new religion dataset** is created:
 - ▶ 819 web pages per class scraped from Atheism and Christianity websites
- Model: SVM with RBF kernel, hyperparameters tuned via cross-validation

🍷 Evaluating with Human Subjects - Can users select the best classifier? ---

- Train two SVMs:
 - ▶ one on problematic dataset
 - ▶ one on “cleaned” dataset
- Recruit humans to select which algorithm will perform best
- The problematic model has *higher* validation accuracy (94.0% vs 88.6%), so accuracy alone would lead to the wrong choice.

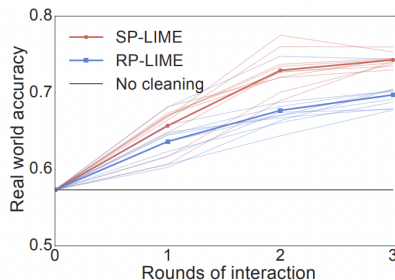


SP-LIME (89%) outperforms RP-LIME (75%), and both outperform greedy variants.

🍷 Evaluating with Human Subjects - Can non-experts improve a classifier? ---

Each round: user sees $B = 10$ instances with $K = 10$ words per explanation and marks words to remove. Model is retrained without those words.

- Round 0: 10 subjects $\rightarrow$ 10 classifiers
- Round 1: 5 new users per classifier $\rightarrow$ 50 classifiers
- Round 2: 5 more users $\rightarrow$ 250 classifiers
- Real-world accuracy measured at each round on the religion dataset



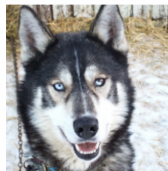
Result

Non-experts using LIME explanations can meaningfully improve a classifier without ever seeing the test data.

🍎 Evaluating with Human Subjects - Do explanations lead to insights? ---

Setup: a deliberately bad classifier is trained to predict “wolf” when there is snow in the background, and “husky” otherwise — ignoring the actual animal.

- ① Find pictures such that classifier predicts “wolf” if there is snow, and “husky” otherwise
- ② Ask subjects (grad students):
 - ▶ Do they trust model to work well in real world?
 - ▶ Why?
 - ▶ How do they think the model distinguishes?
- ③ Showed explanations and asked again



(a) Husky classified as wolf



(b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the “Husky vs Wolf” task.

	Before	After
Trusted the bad model	10 out of 27	3 out of 27
Snow as a potential feature	12 out of 27	25 out of 27

🍷 Related Work ---

Inherently Interpretable Models

- Trees, lists, sets, (generalized) linear models, and additive models
- *Interpretable Decision Sets* (Lakkaraju et al.) — joint framework for description and prediction

Post-hoc, Model-agnostic

- LIME: one of the **first significant papers** for post-hoc, model-agnostic analysis
- *Extracting Faithful and Descriptive Representations* — approximating globally through gradient is a challenge $\Rightarrow$ Local fidelity

Show, Attend and Tell (Xu et al.)

Interpretable description of images via visual attention — LIME aims for a more model-agnostic approach.

🍷 Limitations and Future Work ---

- **Feature space complexity:** expected to work on feature spaces that are not too complex
 - ▶ Can similar ideas be applied to sequential decision making (e.g., RL)?
- **User experiment:** results could depend on how the study notified the definition of “trustworthiness” to participants
- **Automation:** a principled way for the algorithm to automatically improve based on its explanation (without humans having to modify the data or algorithm manually)

🍷 A Unified Approach to Interpreting Model Predictions ---

Scott Lundberg and Su-In Lee

Presented by Max Nadeau, Max Li, and Xander Davies

(Lundberg and Lee 2017)

Outline ---

① Additive Explanations

- ▶ Overview and relation to LIME
- ▶ LIME Desiderata

② Shapley Values

- ▶ Introduction to Shapley values
- ▶ Removing features

③ Approximations

④ Experiments

⑤ Extensions

- ▶ Global interpretability
- ▶ Inner interpretability

🍷 Introduction to Additive Feature Attribution Methods ---

This paper unifies **6 previous methods** for local interpretability as **additive feature attribution methods (AFAMs)**.

An AFAM consists of:

- An enumeration of the **features present** in x
- A protocol for **“removing some features”** from x , defining y (the “input with no features”)
- An approximation $g(x')$ of $f(x)$ and $g(y')$ of $f(y)$
- A partition of $g(x') - g(y')$ among the features of x , indicating **how important each feature was** for the model's output

The importance of feature i is denoted ϕ_i

🍷 LIME as an Additive Feature Attribution Method ---

- LIME (for explaining a classification of some image x) is an AFAM
- The **set of superpixels** is the set of features of x
- We **remove a superpixel** (feature) by replacing its pixels with grey
 - ▶ The image with no features is all grey
- LIME outputs a function g that approximates $f(x)$ and $f(y)$ as $g(x')$ and $g(y')$
- g provides a weighting g_i for the importance of each superpixel in determining $f(x)$ — these serve as the ϕ_i

🍷 DeepLIFT as an Additive Feature Attribution Method ---

- DeepLIFT is another local interpretability method (Shrikumar et al., 2019)
- The **set of pixels** is the set of features of x
- Removing a feature consists of setting a pixel to the value of that pixel in a **reference image** (serves as y)
- DeepLIFT uses $g(x') = f(x)$ and $g(y') = f(y)$ directly (no approximation)
- DeepLIFT calculates a value $C_{\Delta x_i \Delta o}$ for each pixel x_i such that $C_{\Delta x_i \Delta o} = f(x) - f(y)$
 - ▶ Each $C_{\Delta x_i \Delta o}$ represents importance of x_i to classification $f(x)$

🍷 Desiderata for Additive Feature Attribution Methods ---

Three desirable properties proposed for an AFAM:

Local Accuracy

$g(x') = f(x)$. DeepLIFT meets this; LIME does not necessarily.

Consistency

For input x , let $x \setminus i$ denote "removing feature i from x ". If including feature i in the input always makes a bigger difference in model f than in model f' , then the AFAM should give higher importance ϕ_i for model f than for f' .

Missingness

Described as "really just a minor book-keeping property" — we'll ignore it.

Cooperative Games ---

- Suppose we have a game with d players, where each can choose whether or not to cooperate
- Assume a **reward function** $g : \mathcal{P}([d]) \rightarrow \mathbb{R}$. If $S \subseteq [d]$ is the set of cooperating players, the group receives reward $g(S)$
- We want to determine how much each player **“contributes”** to the reward
 - ▶ The marginal contribution of player i may depend on which other players have also chosen to cooperate
 - ▶ g does not need to be monotonic!

🍷 Shapley Values ---

The **Banzhaf power index** averages player i 's marginal contribution over all subsets:

$$\frac{1}{2^{d-1}} \sum_{S \subseteq [d] \setminus \{i\}} (g(S \cup \{i\}) - g(S)) \quad (1)$$

The **Shapley value** reweights marginal contributions based on the size of subset S :

$$\frac{1}{d} \sum_{j=0}^{d-1} \binom{d-1}{j}^{-1} \sum_{S \subseteq [d] \setminus \{i\}, |S|=j} (g(S \cup \{i\}) - g(S)) \quad (2)$$

Or equivalently: $\sum_{\sigma \in S_d} g(\sigma(\sigma(i))) - g(\sigma(\sigma(i) - 1))$

🍷 From Games to Local Interpretability ---

- Suppose for some input x , a model produces prediction $f(x)$, and we want to measure how “important” each feature was
- We can treat the **features as players** in a cooperative game, and ask how much each contributed to the output
- However, to prompt the model, we need to provide all input features S . How do we measure what the model would have predicted **if it only had access to a subset of features**?

In other words, the function $g : \mathcal{P}([d]) \rightarrow \mathbb{R}$ is not well-defined.

🍷 Previously Proposed: Separate Models ---

- In “**Shapley regression values**”, for every subset of features $S \subseteq [d]$, we can train a model f_S that tries to predict the labels
- The resulting Shapley values are a good metric for how important each feature is for good prediction of the labels
- However, they do **not** provide interpretability for the specific model f we were working with
 - ▶ What f might do without any information about the features in $[d] \setminus S$ might be very different than optimal prediction

🍷 Feature Ablation ---

We want to capture what our **particular** model f would do if it had no access to features $[d] \setminus S$.

This notion is captured by the **expectation of the model output given features S** :

$$E[f(x) \mid x_S] \quad (3)$$

We can approximate this by sampling over the conditional distribution:

$$\frac{1}{N} \sum_{i=1}^N f(x^{(i)}), \quad x^{(i)} \sim x \mid x_S \quad (4)$$

🍷 Model Agnostic: Feature Independence ---

- Computing SHAP values requires calculating 2^d differences $g(S \cup \{i\}) - g(S)$
- One way to improve: assume **features are independent**, $\forall S$:

$$E[f(x) \mid x_S] = E_{x_S|x_S}[f(x)] \approx E_{x_S}[f(x)] \quad (5)$$

- We can then estimate SHAP values via sampling approximations (**the Shapley sampling values method**) which require fewer than 2^d difference calculations
- But still requires lots of computations

🍷 Model Agnostic: Kernel SHAP via LIME ---

The kernel weights (π_x), loss function (L), and simplicity function (Ω) in LIME aren't consistent with desired properties (local accuracy & consistency). We adjust these to form the **Shapley kernel**:

$$\Omega(g) = 0 \quad (6)$$

$$\pi_x(z') = \frac{(M-1)}{\binom{M}{|z'|} |z'| (M - |z'|)} \quad (7)$$

$$L(f, g, \pi_x) = \sum_{z' \in \mathcal{Z}} [f_h(z') - g(z')]^2 \pi_x(z') \quad (8)$$

where $|z'|$ is the number of non-zero elements in z' .

🍷 Kernel SHAP: Sample Weight Comparison ---



🍷 Model Agnostic: Kernel SHAP (cont.) ---

We can consider LIME's feature removing protocol an **approximation of SHAP values** that assumes f is linear, so that $E[f(x) \mid x_S] = f(x^*)$, where:

$$x_i^* = \begin{cases} x_i & \text{if } i \in S \\ E[x_i] & \text{otherwise} \end{cases} \quad (9)$$

- Since we can solve for g in (8) as a **weighted linear regression problem**, we have a regression-based, model-agnostic estimation of SHAP values!
- This is **more efficient** than previously, since we jointly solve for SHAP values

🍷 Model-Specific Approximations: Linear SHAP ---

We can do better by looking for **model-specific approximations**.

If the model is **affine** and we assume **feature independence**, feature i 's importance for x is its difference from the mean multiplied by its weight:

If $f(x) = \sum_{j=1}^M w_j x_j + b$, then:

$$\phi_0(f, x) = b$$

$$\phi_i(f, x) = w_i(x_i - E[x_i])$$

🍷 Model-Specific Approximations: Deep SHAP ---

DeepLIFT approximates SHAP values assuming **feature independence** and the deep model is **linear**, since it:

- ① **Linearizes** the non-linear components of a network (“heuristically chosen”)
- ② **Replaces values** with a reference value, which we can consider $E[x]$ (like LIME)
 - Currently satisfies **local accuracy** (and missingness), but **not consistency**
 - We can choose **new linearizations** which satisfy consistency

⇒ **Deep SHAP!**

🍷 User Experiments ---

- The authors run an experiment in which they tell a story about people playing a **cooperative game**, and find that **human assignments of credit align better with SHAP's assignment** than with LIME's or DeepLIFT's

Caveat

This is a very different setting from attribution in neural networks, seemingly selected to make SHAP look good, so this experiment is unimpressive evidence that SHAP aligns with human intuition for NN credit assignment.

🍷 Class Difference Experiments ---

- Using an image of an "8" and an MNIST classifier, the authors identified which pixels (according to SHAP, LIME, and DeepLIFT) are most important for the model's log-odds (i.e. logit difference) of 8 versus 3 - **Removing the pixels identified by SHAP** produced larger changes in log-odds from 8 to 3 than the other methods



🍷 Extensions: Shapley Values for the Whole Model ---

Rather than attributing Shapley values for a particular input x , we can instead attribute Shapley values for the **model's prediction over the entire input distribution**.

- Naively: $g(S) = E[E[f(x) \mid x_S]]$, which reduces to $E[f(x)]$ by Adam's law
- Instead, to capture the amount of model behavior we can explain with only a subset of features, use a **symmetric loss function**:

$$g(S) = \text{Var}(E[f(x) \mid x_S])$$

Methods that do this include **SAGE** (Covert et al., 2020) and **Shapley Effects** (Owen, 2014).

🍷 Extensions: Neuron Shapley ---

- We can **treat neurons as features** (Ghorbani and Zou, 2020)
- This paper uses **zero ablation**
- We can also use **multi-armed bandit sampling algorithms** to estimate Shapley values

Neuron Shapley extends SHAP concepts to understand the contribution of individual neurons in neural networks.

Thank You!

References --- |

- Lundberg, Scott M, and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." In *Advances in Neural Information Processing Systems*. Vol. 30.
- Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin. 2016. "'Why Should I Trust You?': Explaining the Predictions of Any Classifier." In *Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 1135–44.